

Modeling Physical Systems using Hamiltonian Neural Networks

Spring-Mass and Pendulum Systems

Presented by:

Amit Kumar & **Prashant Kumar**

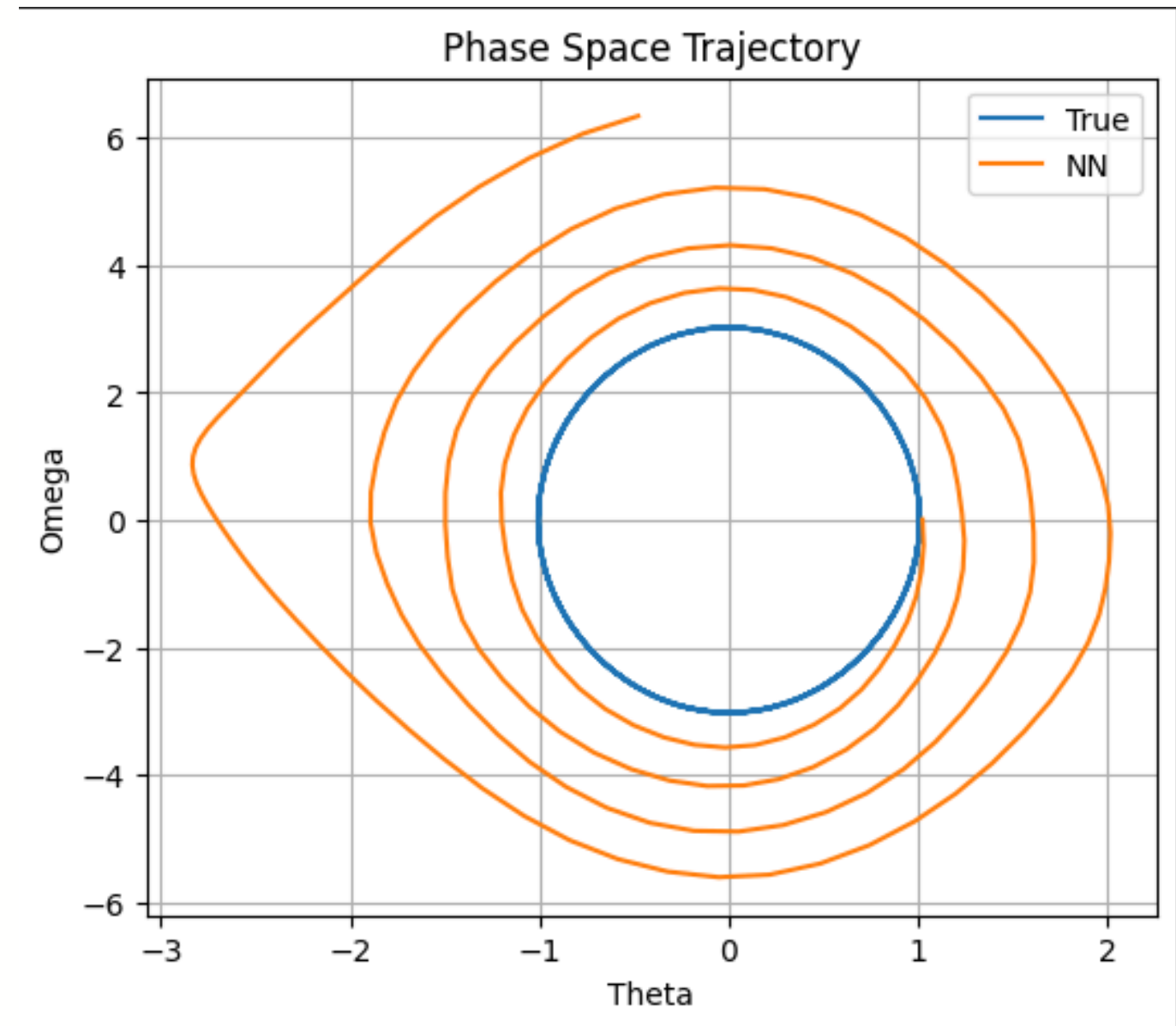
(CS23B1004, CS23B1058)

Department of Computer Science & Engineering
Indian Institute of Information Technology Raichur

Under Guidance of:
Dr. Priodyuti Pradhan

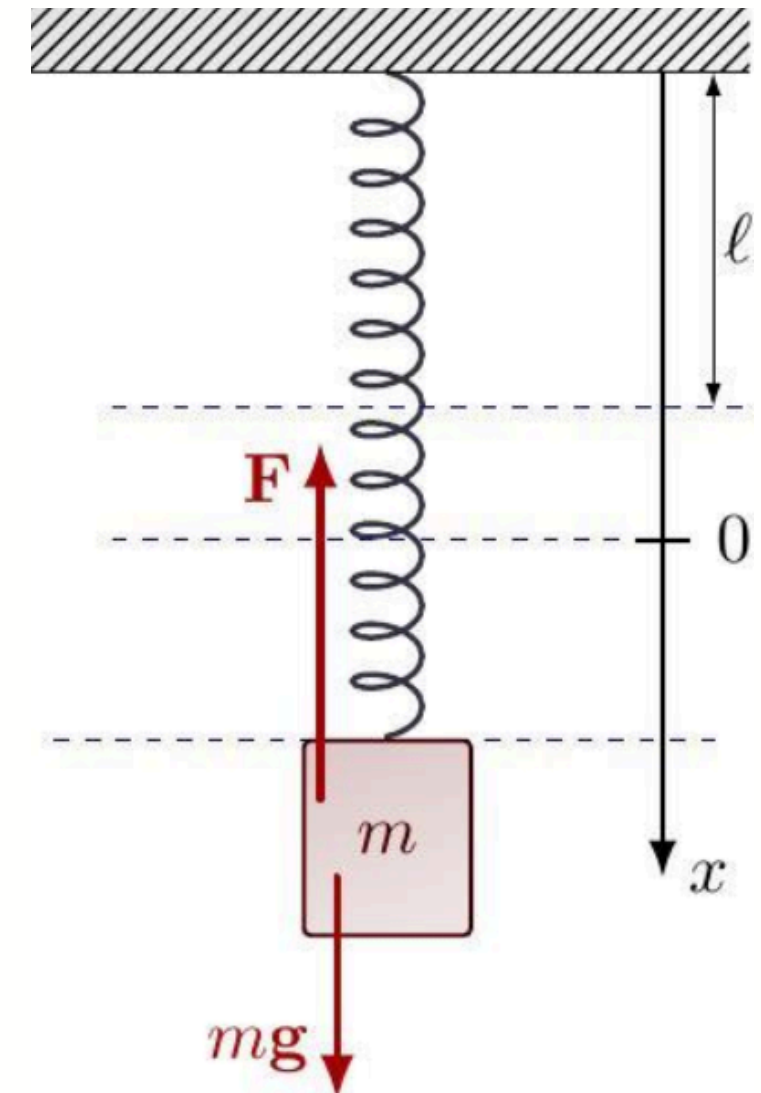
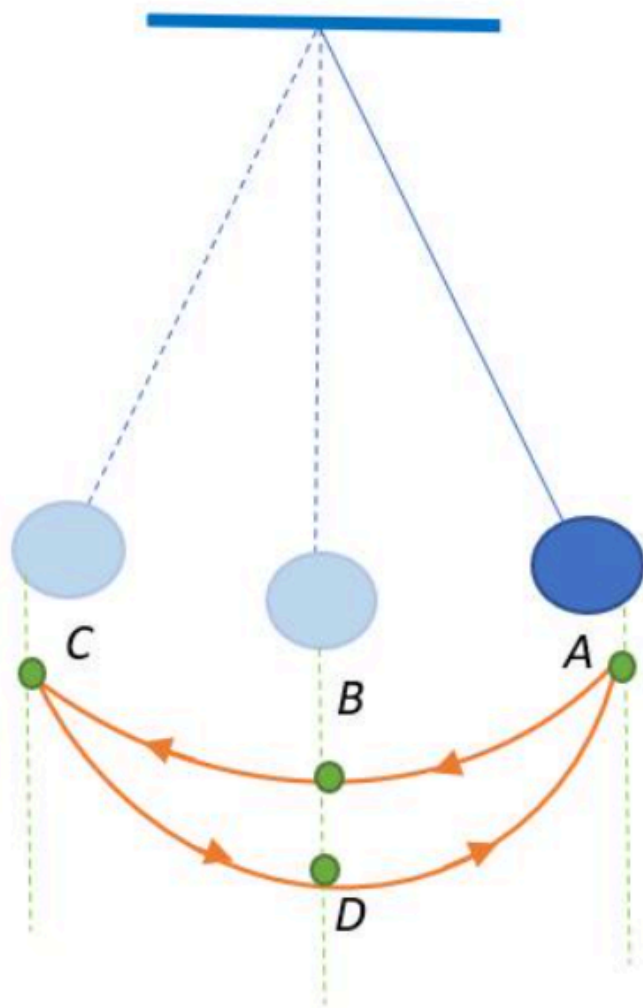
Motivation

- Neural networks learn patterns, not physical laws
- Physical systems obey conservation laws (energy)
- Energy conservation ensures stable and realistic behavior
- Standard NN fails to preserve this
- Errors accumulate over time
- Need models that respect physics



Problem Statement

- Learn dynamics of physical systems from data
- Preserve underlying physical laws (energy conservation)
- Analytical solutions exist for simple systems
- Real-world systems often do not have closed-form solutions
- Systems considered:
 - Spring-Mass System
 - Pendulum System



System Equations

Spring-Mass System

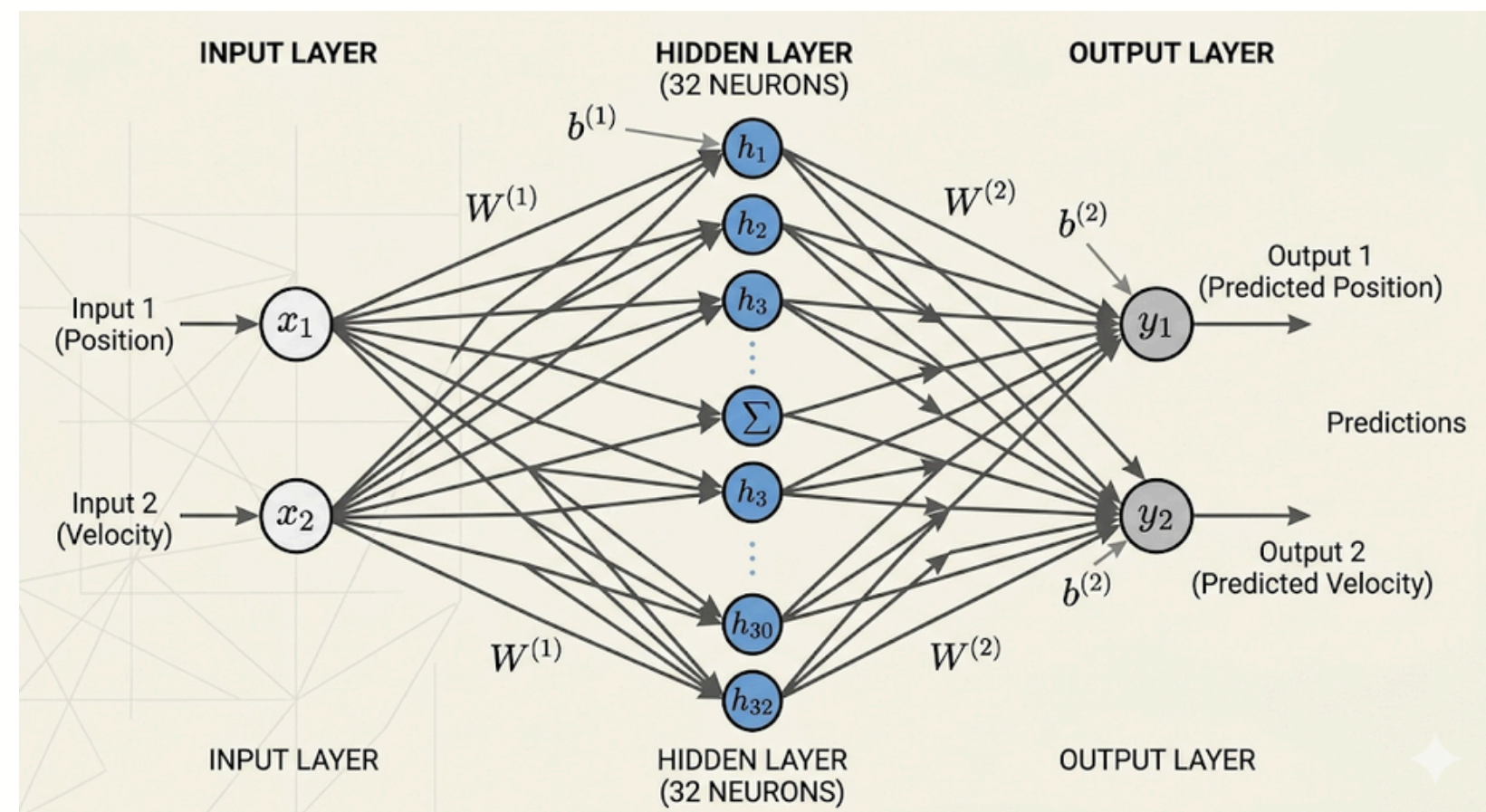
- Equation of motion:
 - $m(d^2x/dt^2) = -kx$
- Components:
 - m : mass
 - k : spring constant
 - x : displacement
- Energy:
 - Kinetic: $\frac{1}{2} mv^2$
 - Potential: $\frac{1}{2} kx^2$

Pendulum System

- Equation of motion:
 - $d^2\theta/dt^2 + g/l\sin\theta = 0$
- Components:
 - θ : angle
 - g : gravity
 - l : length
- Energy:
 - Kinetic: $\frac{1}{2} ml^2\dot{\theta}^2$
 - Potential: $mgl(1-\cos\theta)$

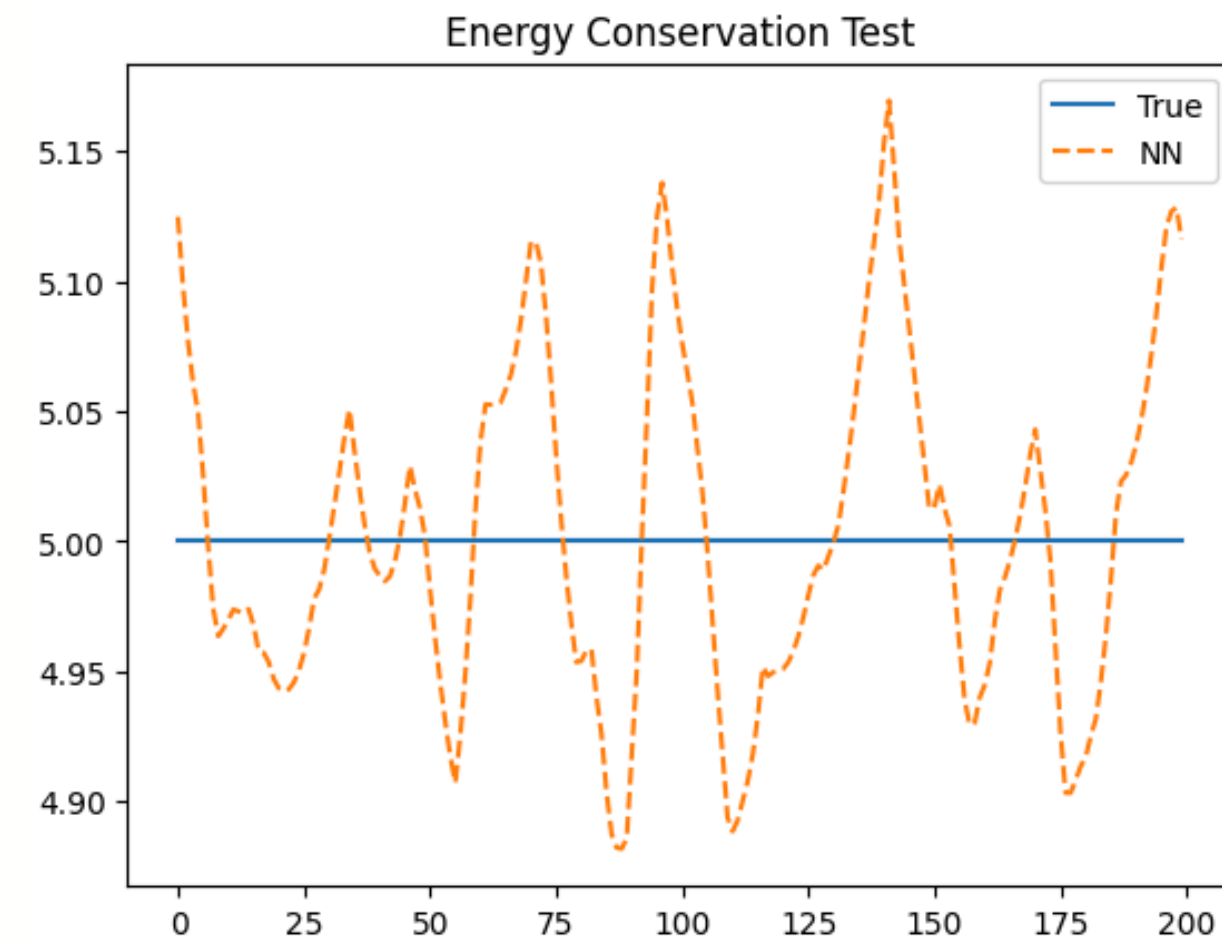
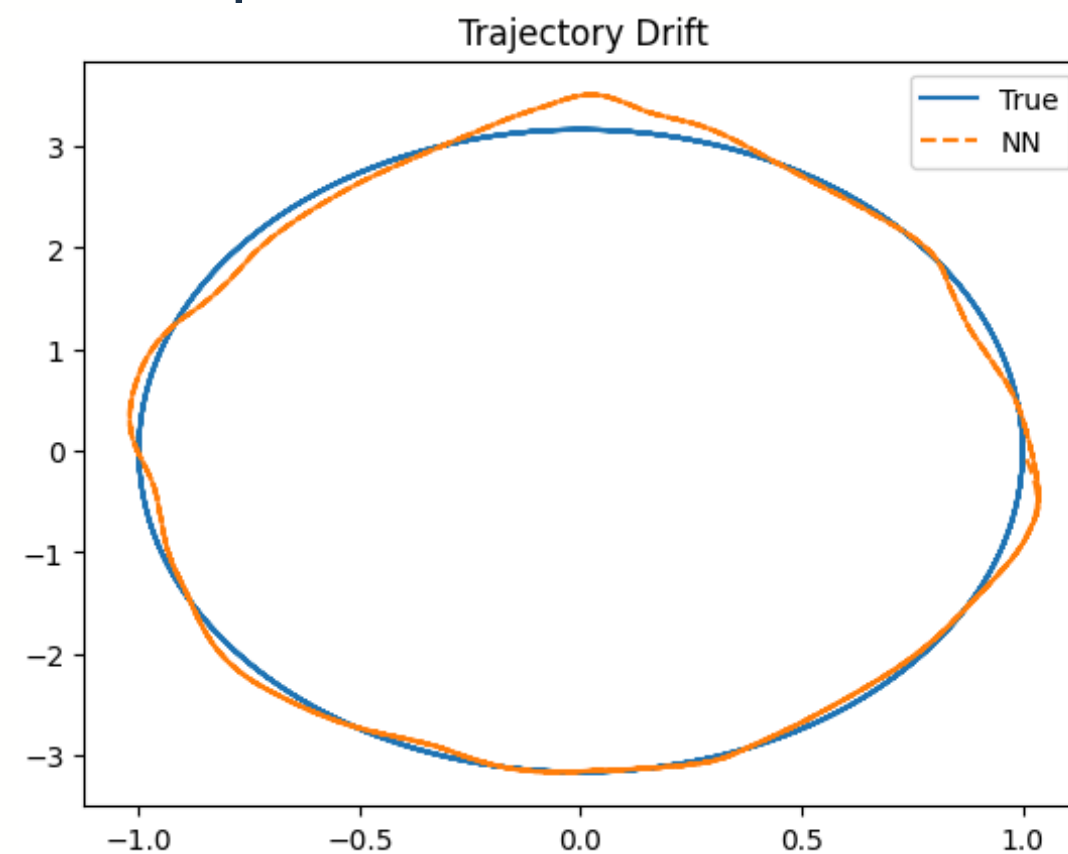
Baseline Neural Network

- State of system:
 - Position (q), Velocity (v)
- Neural network learns:
 - $f(q, v) = \dot{q}, \dot{v}$
- Predicts how the system evolves over time
- No physical constraints enforced



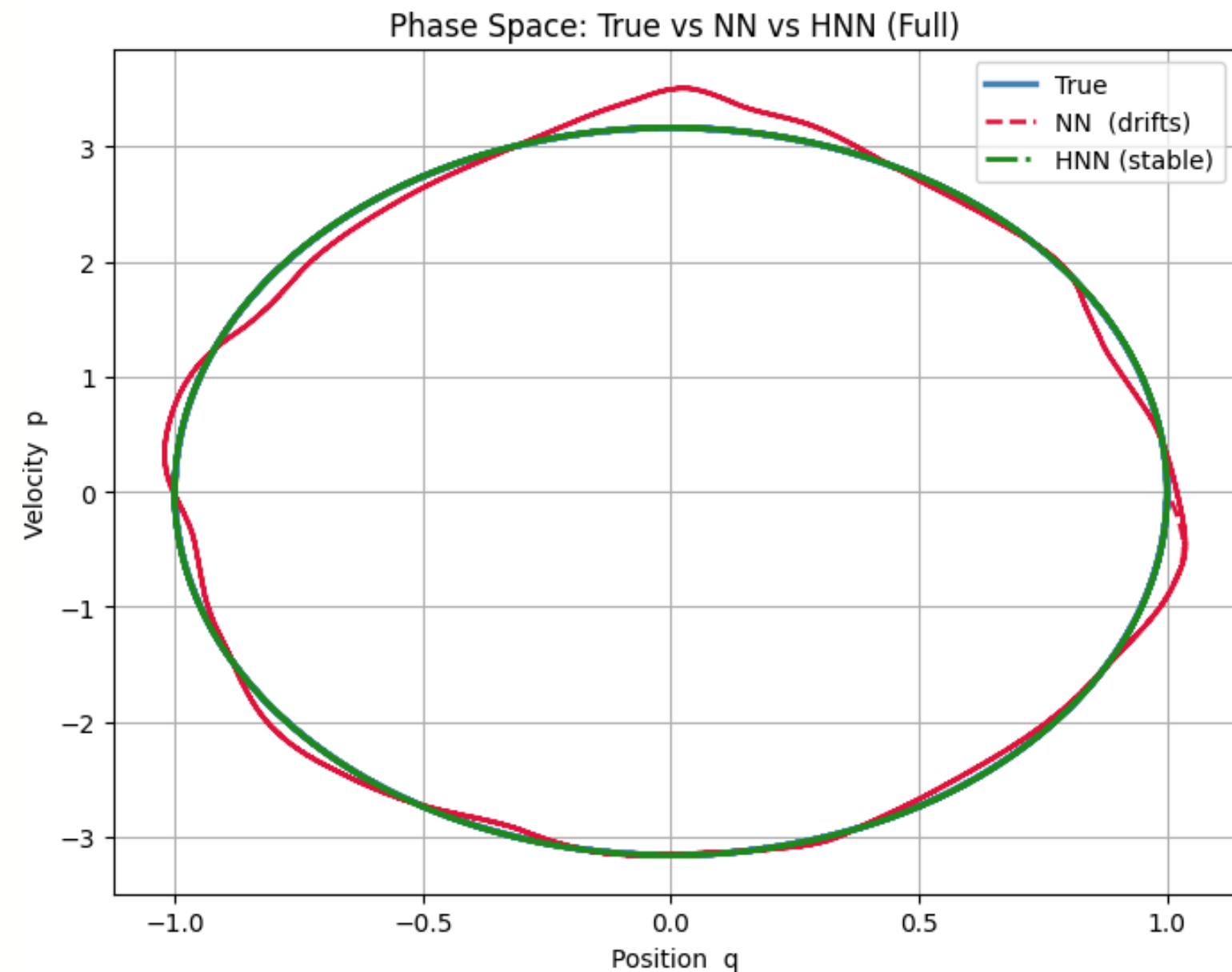
Failure of Baseline NN

- Energy conservation is fundamental in physical systems
- Does not conserve energy
- Errors accumulate over time
- Trajectories diverge from true motion
- Unstable long-term predictions



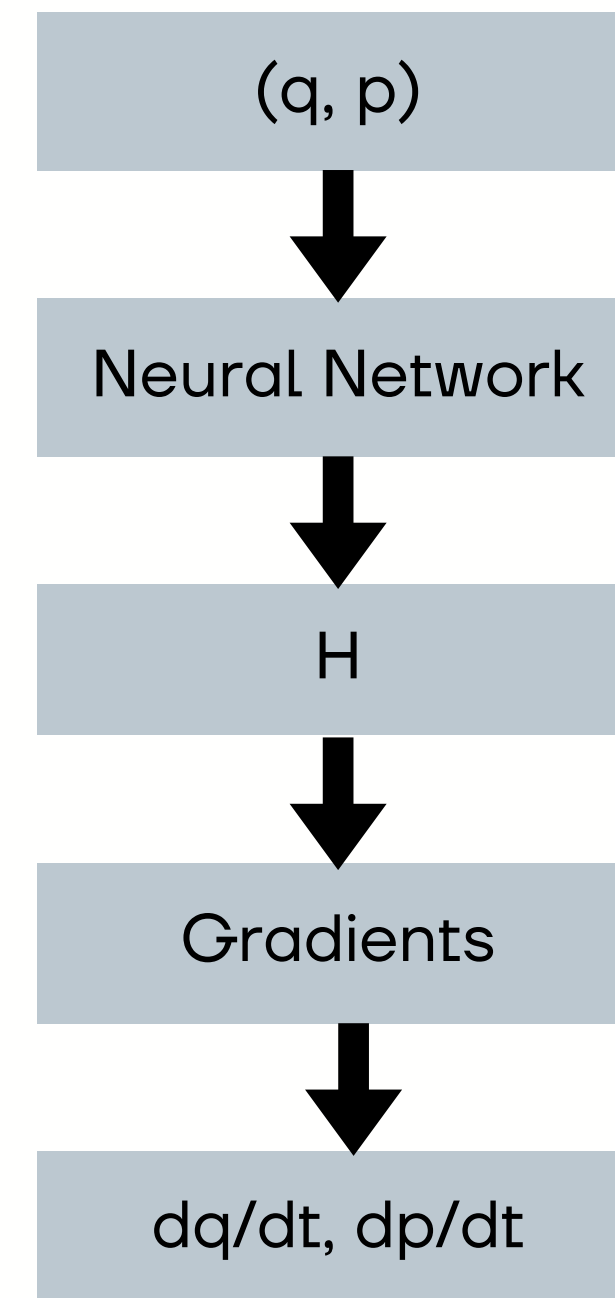
Hamiltonian Mechanics

- State defined by:
 - Position (q), Momentum (p)
- System described by Hamiltonian:
 - $H(q, p)$ = Total Energy
- Governing equations:
 - $\dot{q} = \partial H / \partial p$
 - $\dot{p} = -\partial H / \partial q$
- Energy H is conserved along trajectories
- Phase space trajectories form a closed orbits



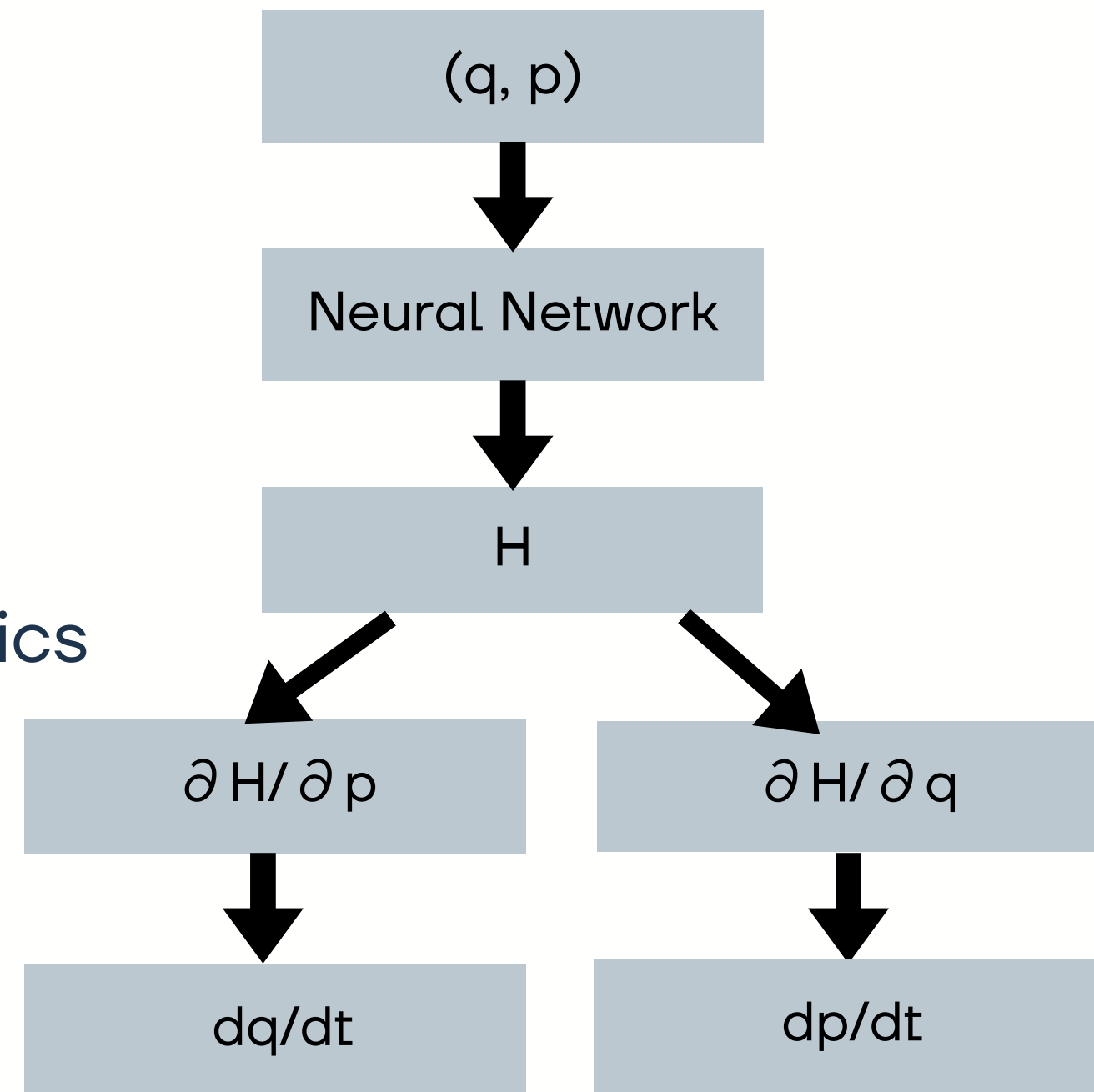
Hamiltonian Neural Network (HNN)

- Neural network learns Hamiltonian $H(q, p)$
- Outputs scalar energy instead of next state
- System dynamics derived using gradients
- Embeds physical laws into the model



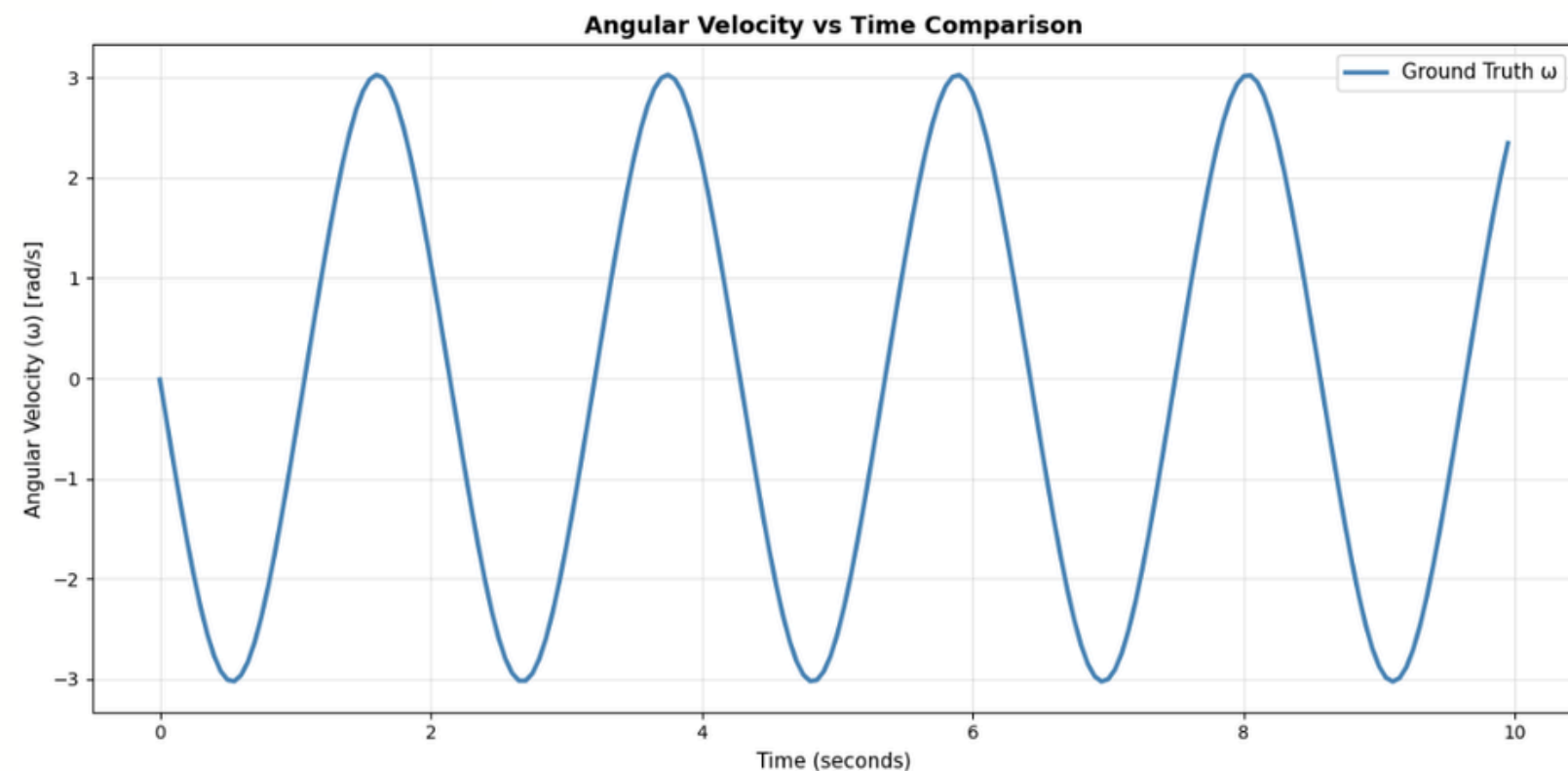
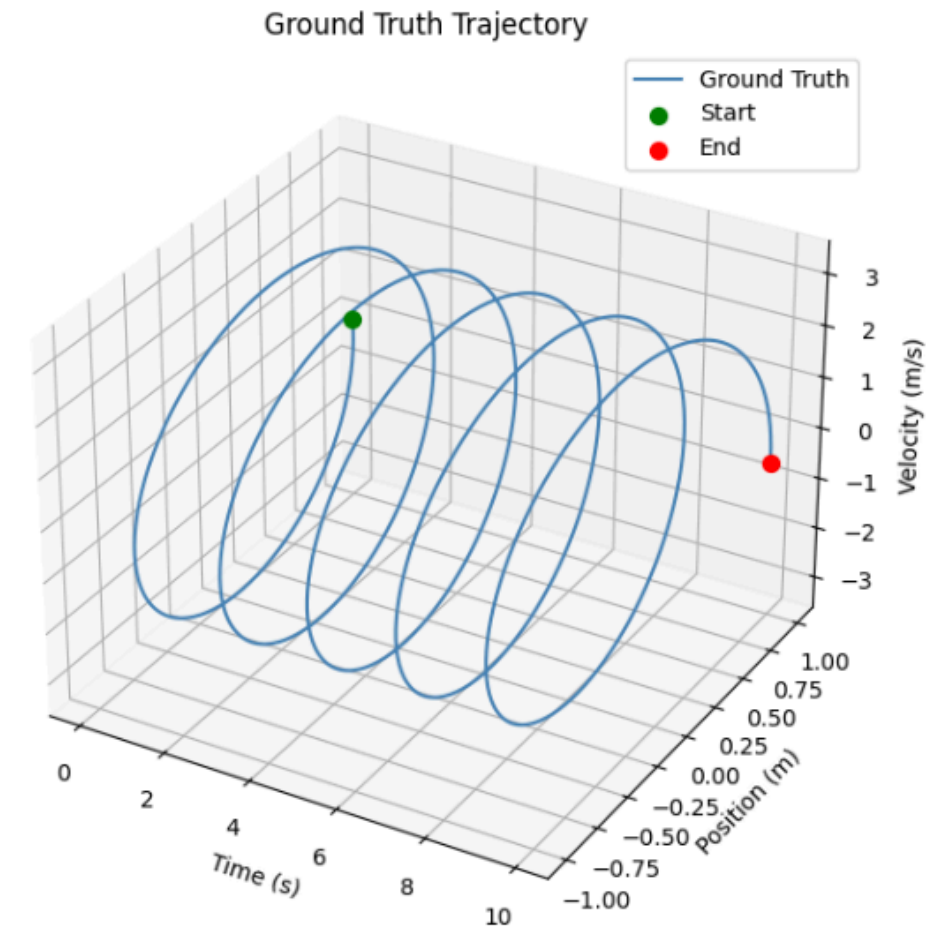
Model Architecture

- Input: position (q), momentum (p)
- Neural network outputs: scalar $H(q, p)$
- Compute gradients:
 - $\partial H / \partial p \rightarrow$ velocity
 - $\partial H / \partial q \rightarrow$ force
- Use Hamilton's equations to get dynamics



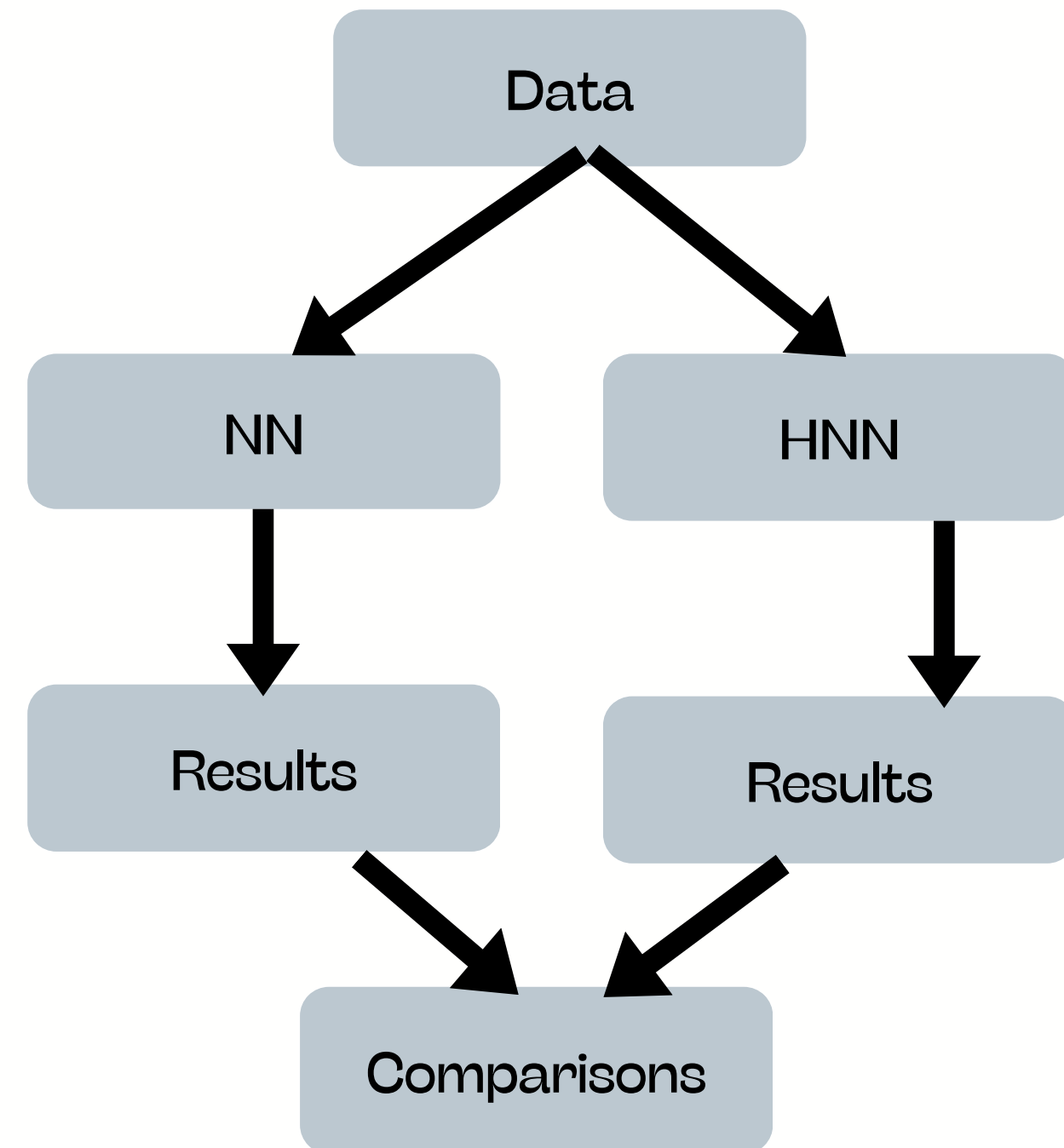
Dataset

- Systems used:
 - Spring-Mass (linear system)
 - Pendulum (non-linear system)
- Data consists of:
 - Position, velocity
 - Time evolution
- Generated using known physical equations



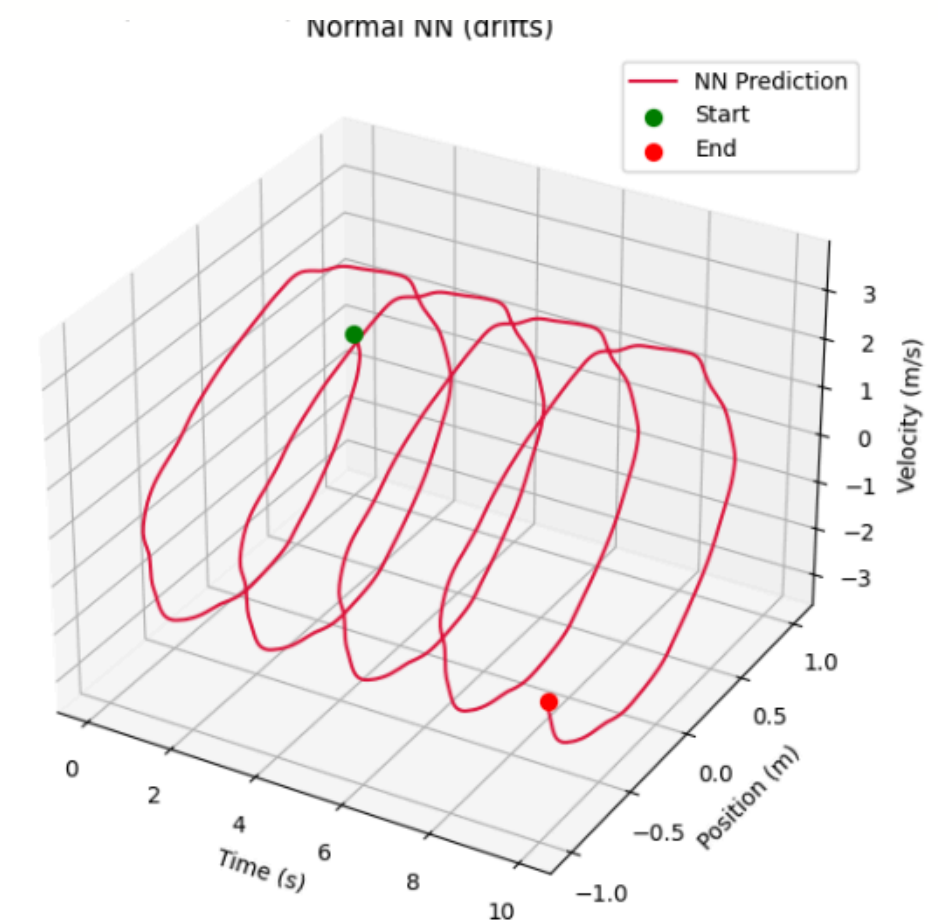
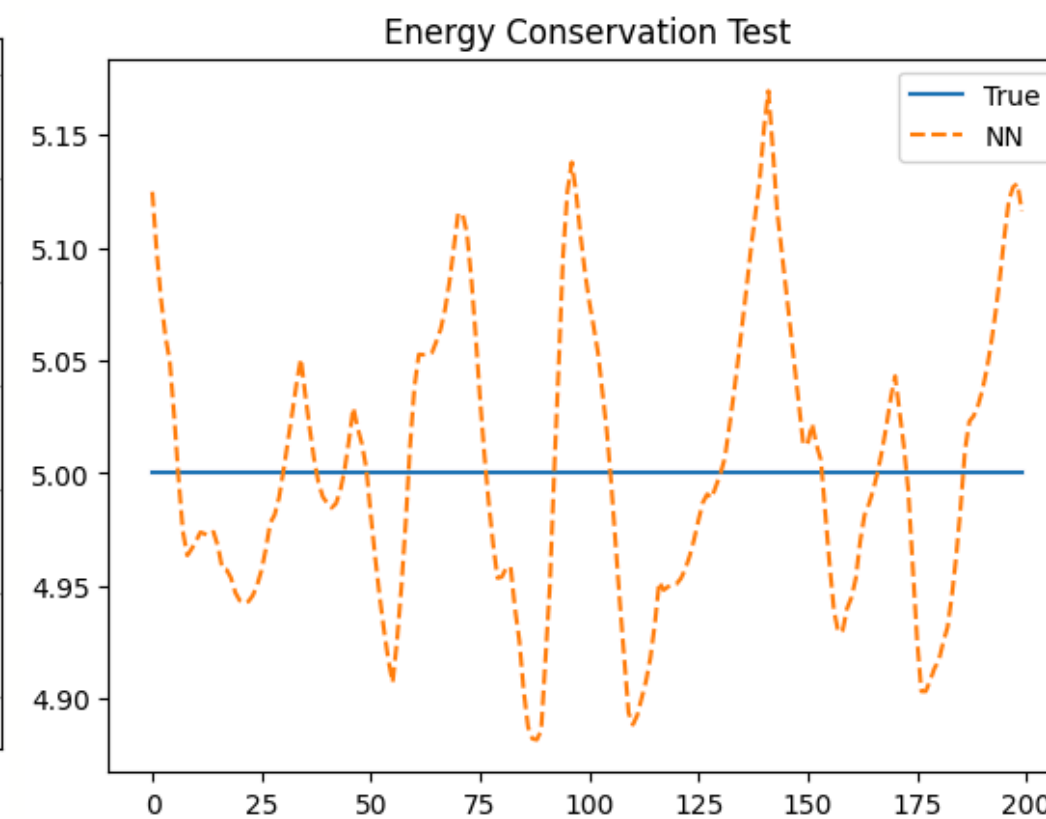
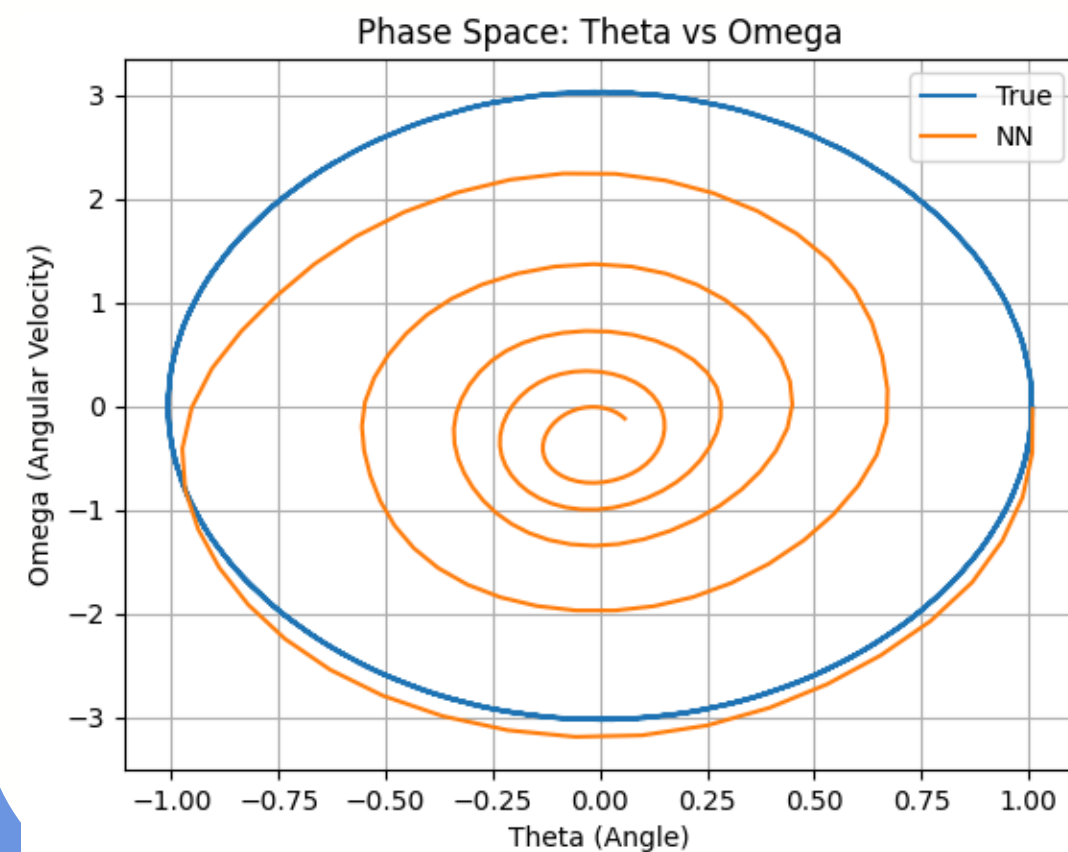
Workflow

- Generate dataset from physical equations
- Train baseline neural network (NN)
- Train Hamiltonian Neural Network (HNN)
- Compare results (trajectory + energy)



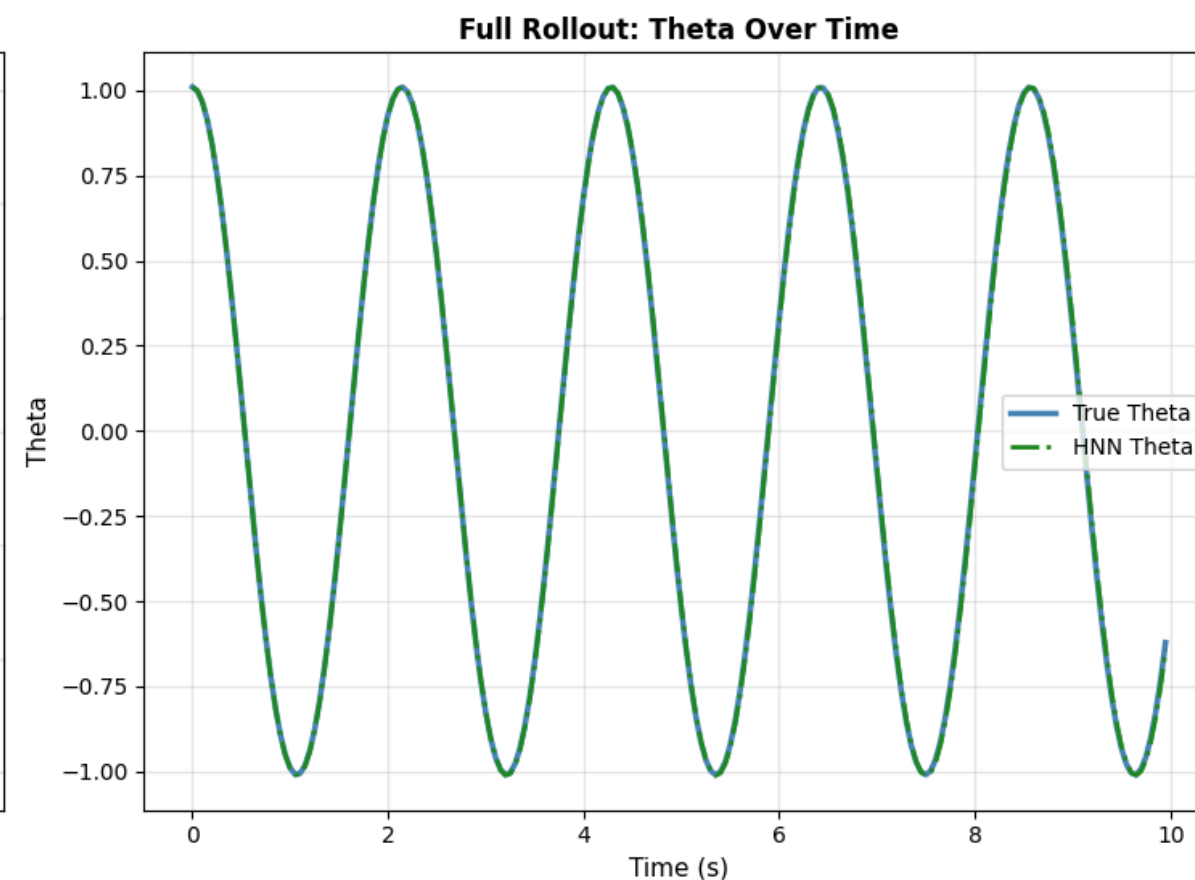
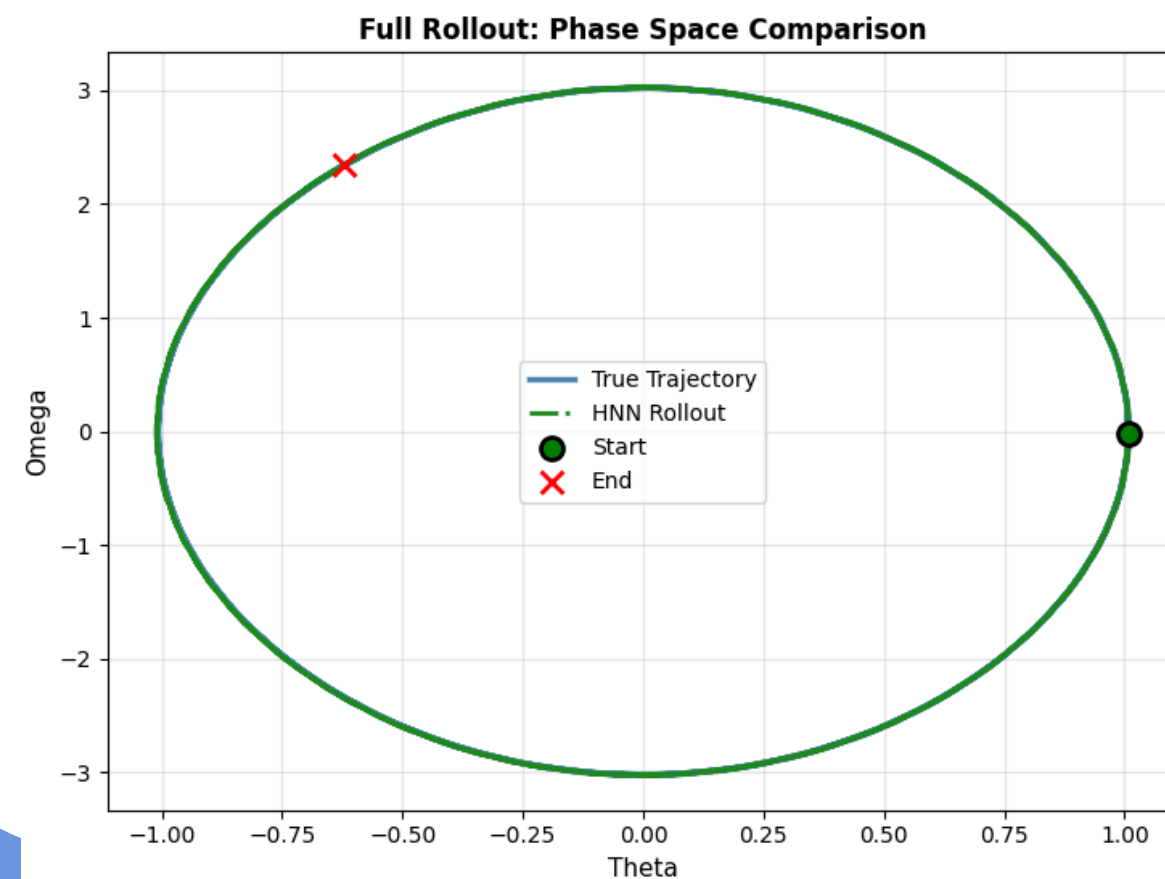
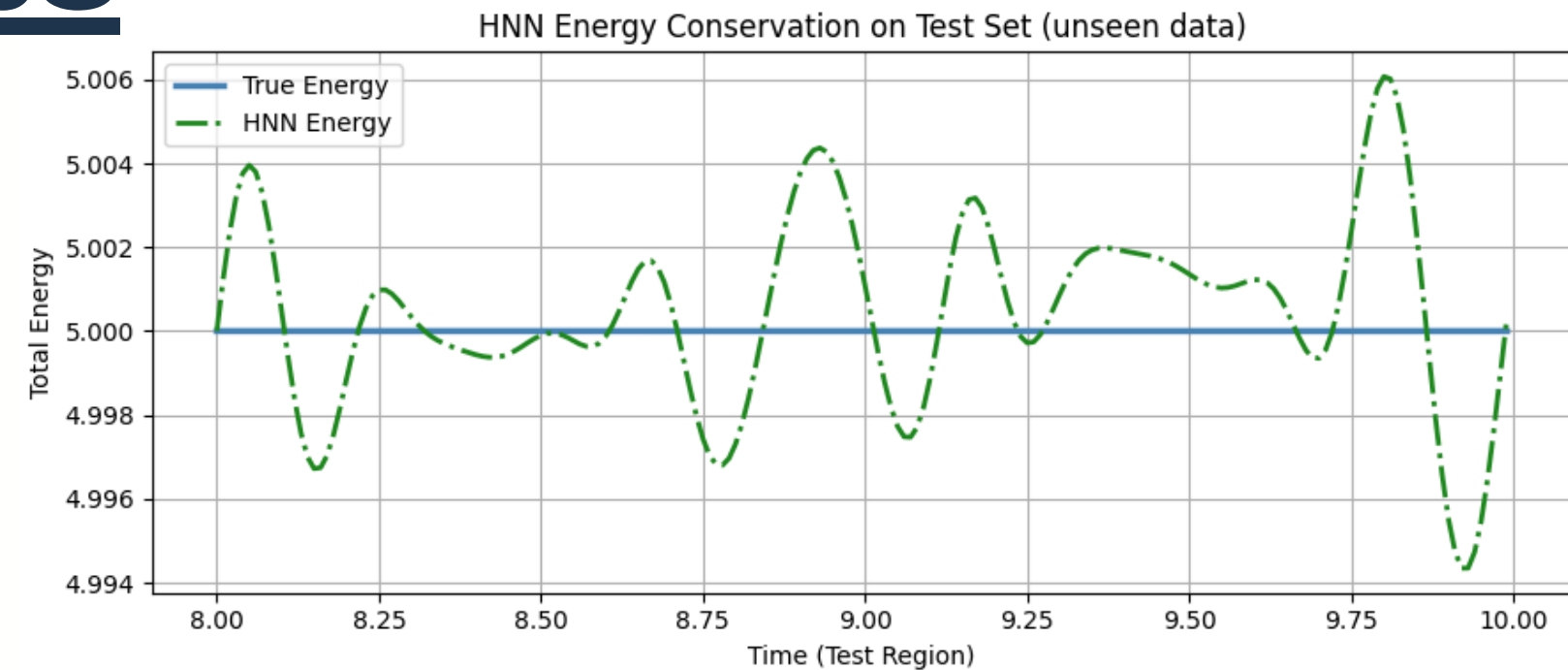
Baseline NN Results

- Accurate in short-term predictions
- Performance degrades over time
- Energy is not conserved
- Trajectory deviates from true motion



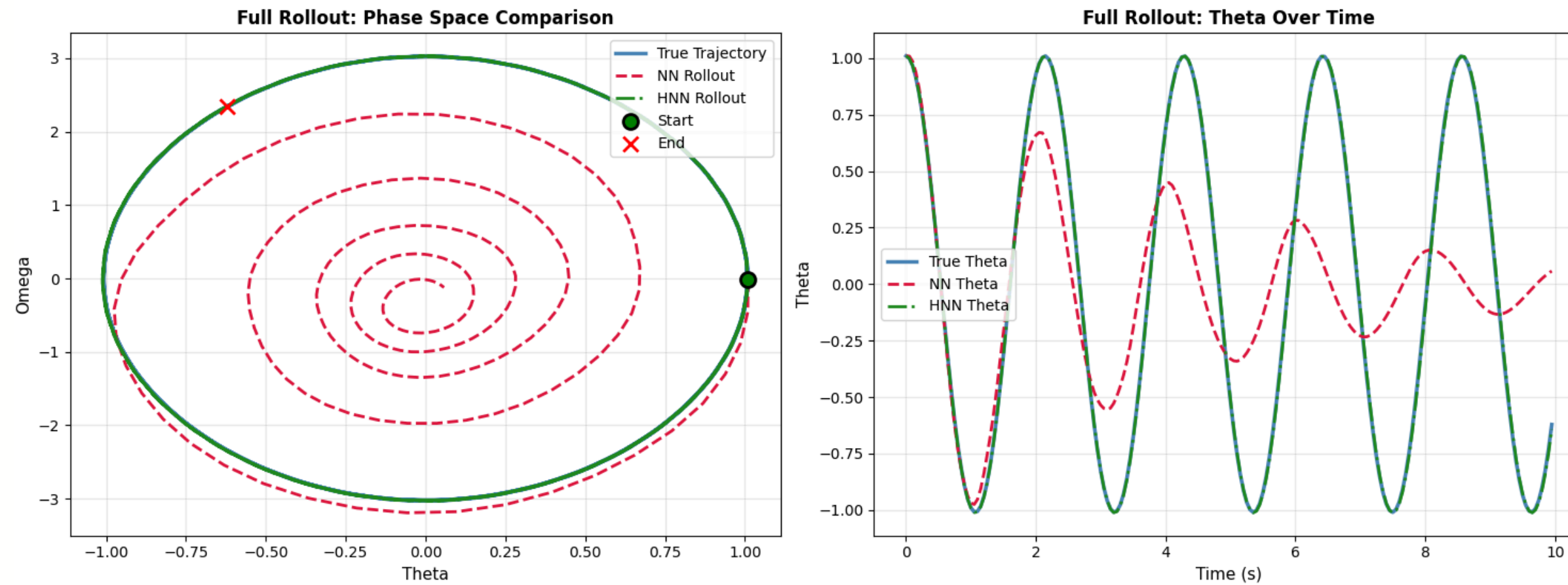
HNN Results

- Stable long-term predictions
- Energy remains conserved
- Trajectories closely match true motion
- Better generalization than NN



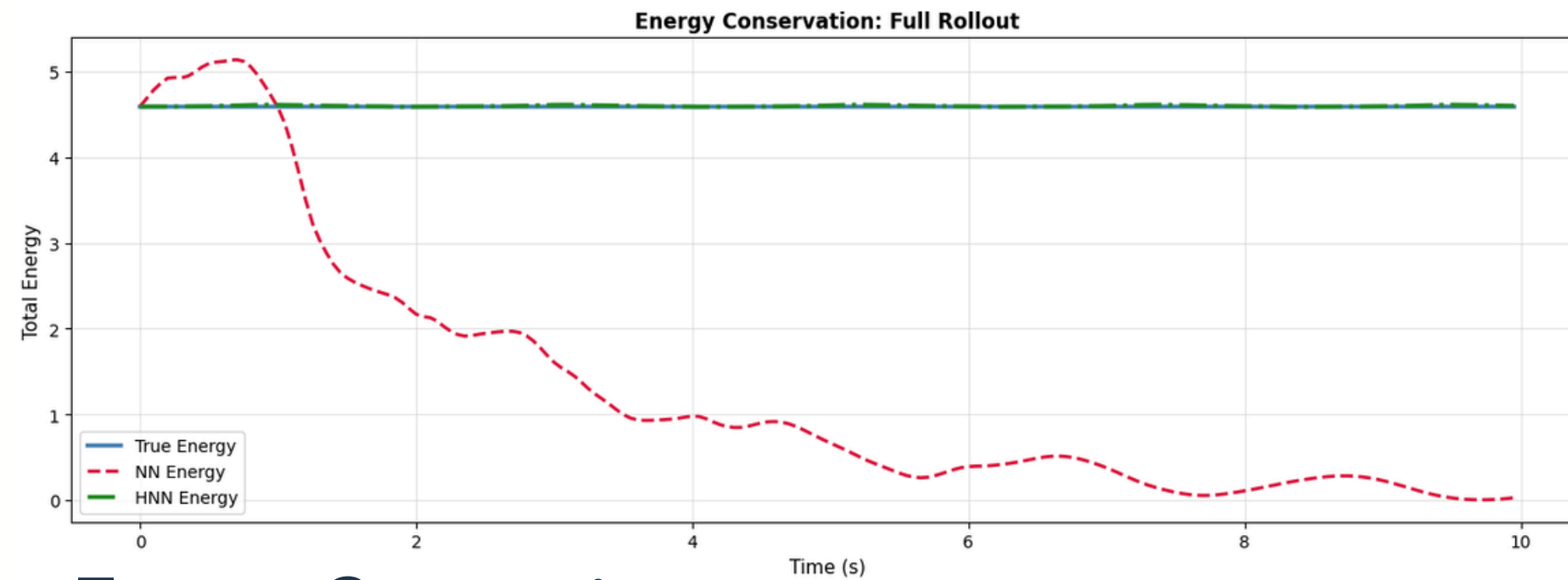
Direct Comparison (NN vs HNN)

Pendulum System



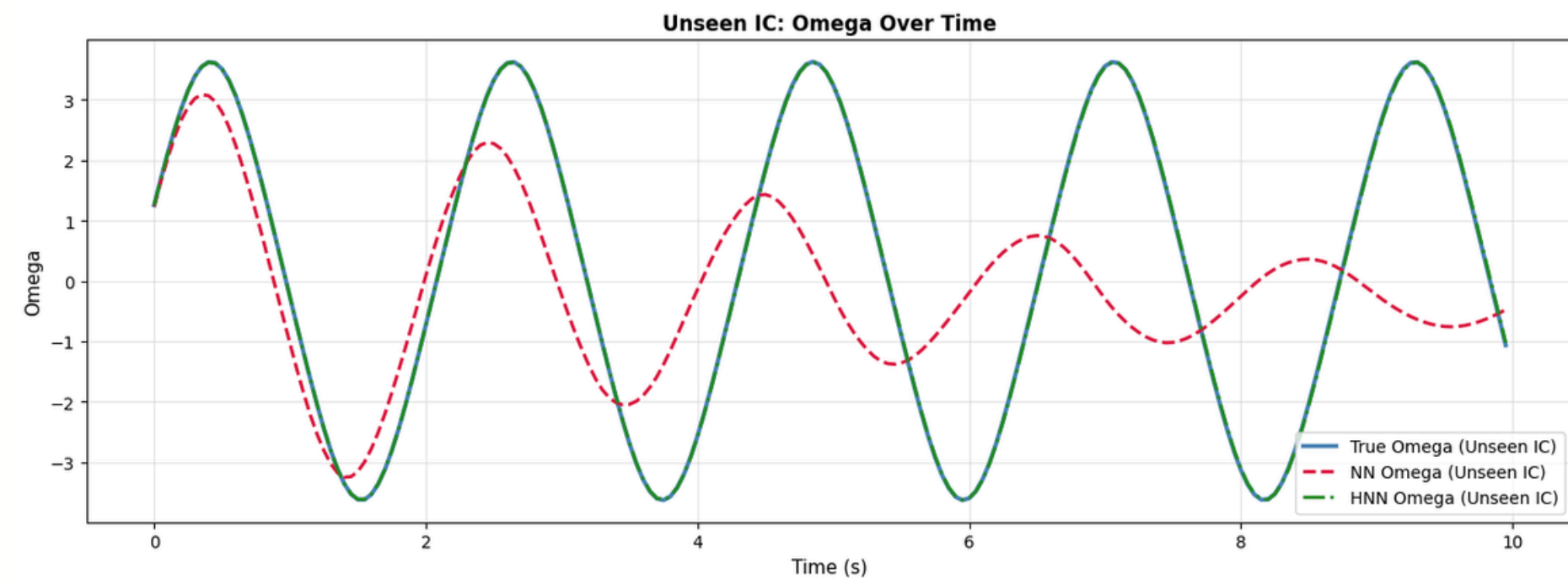
Trajectory Comparison

Direct Comparison (NN vs HNN)



Energy Comparison

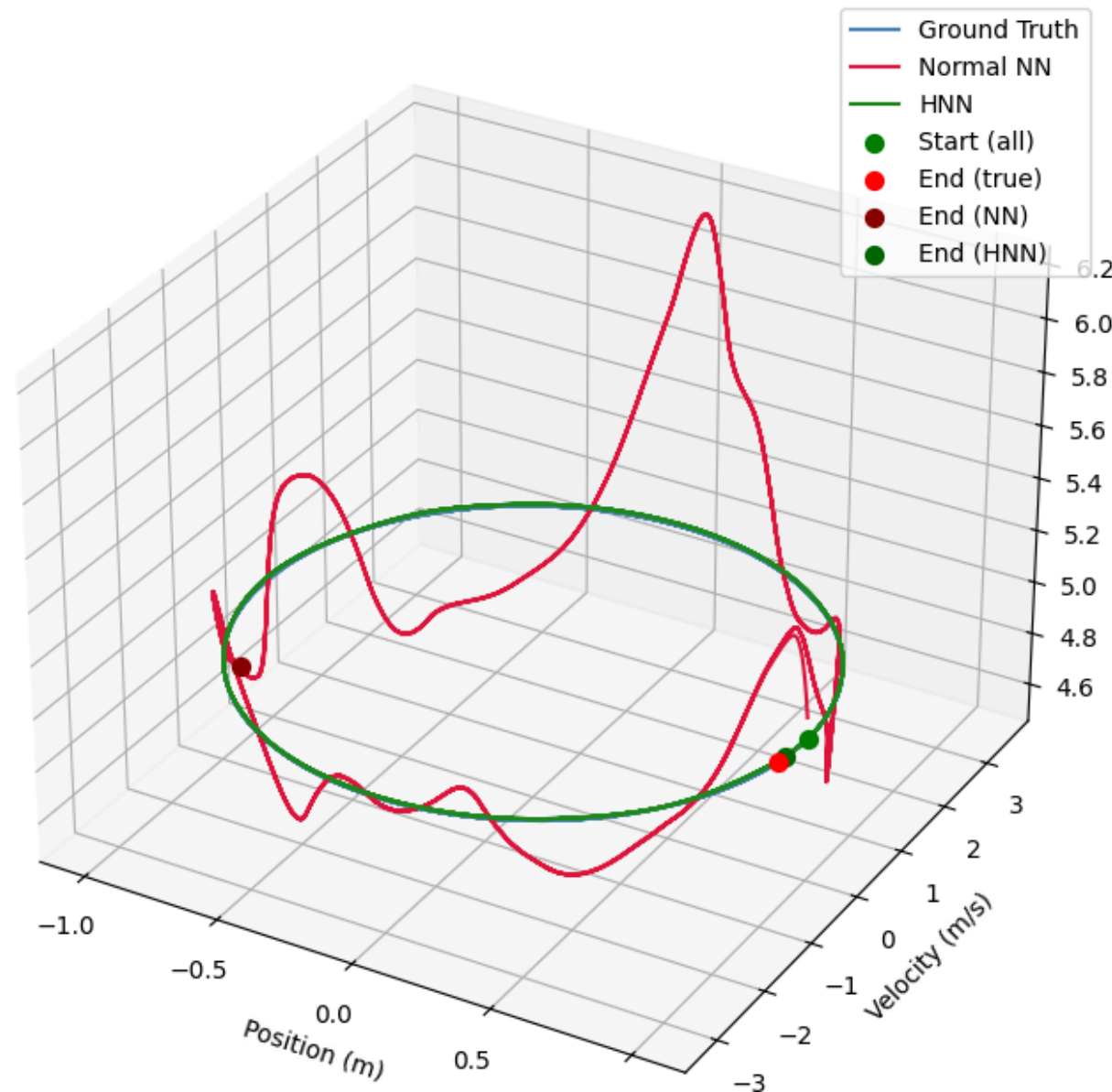
Omega Comparison



Direct Comparison (NN vs HNN)

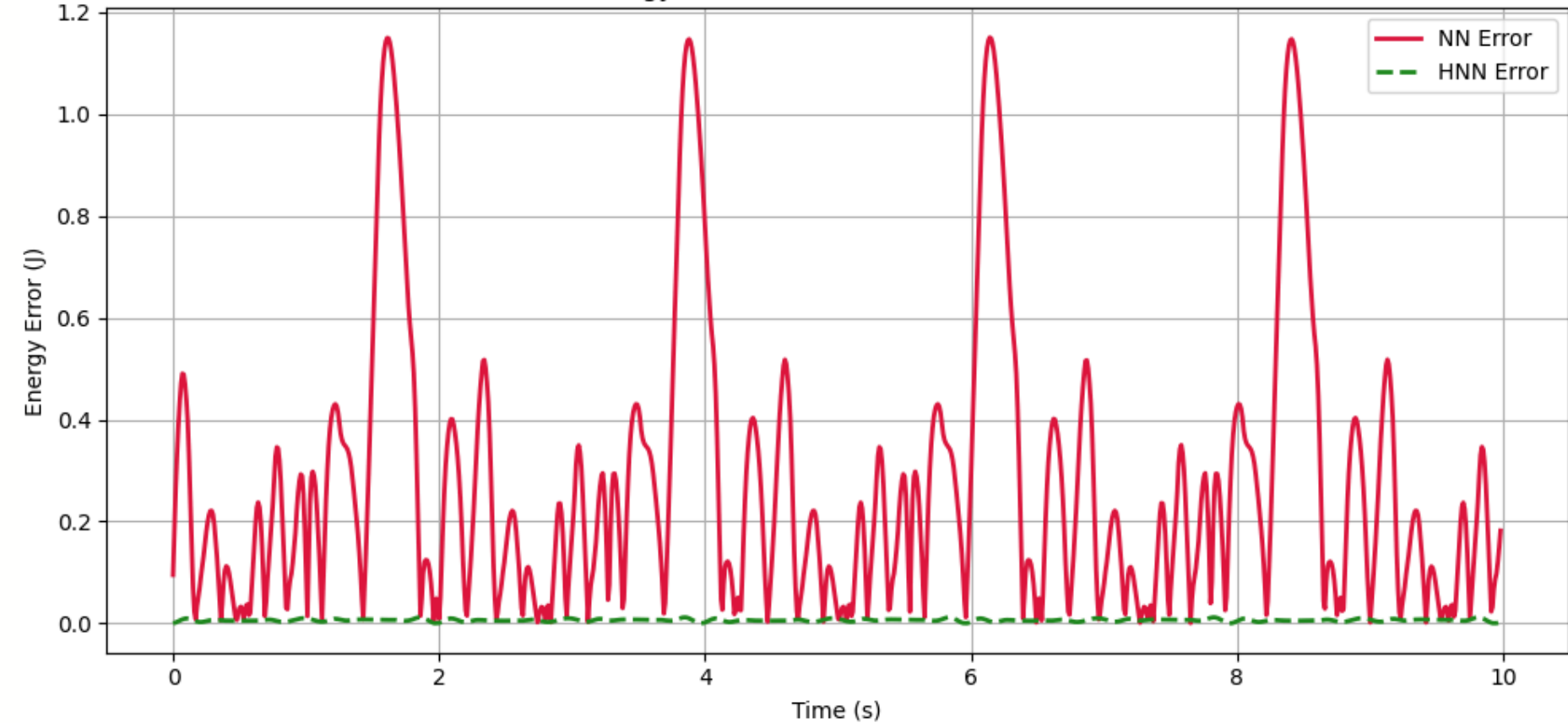
Spring Mass System

Full Comparison: Position-Velocity-Energy Space



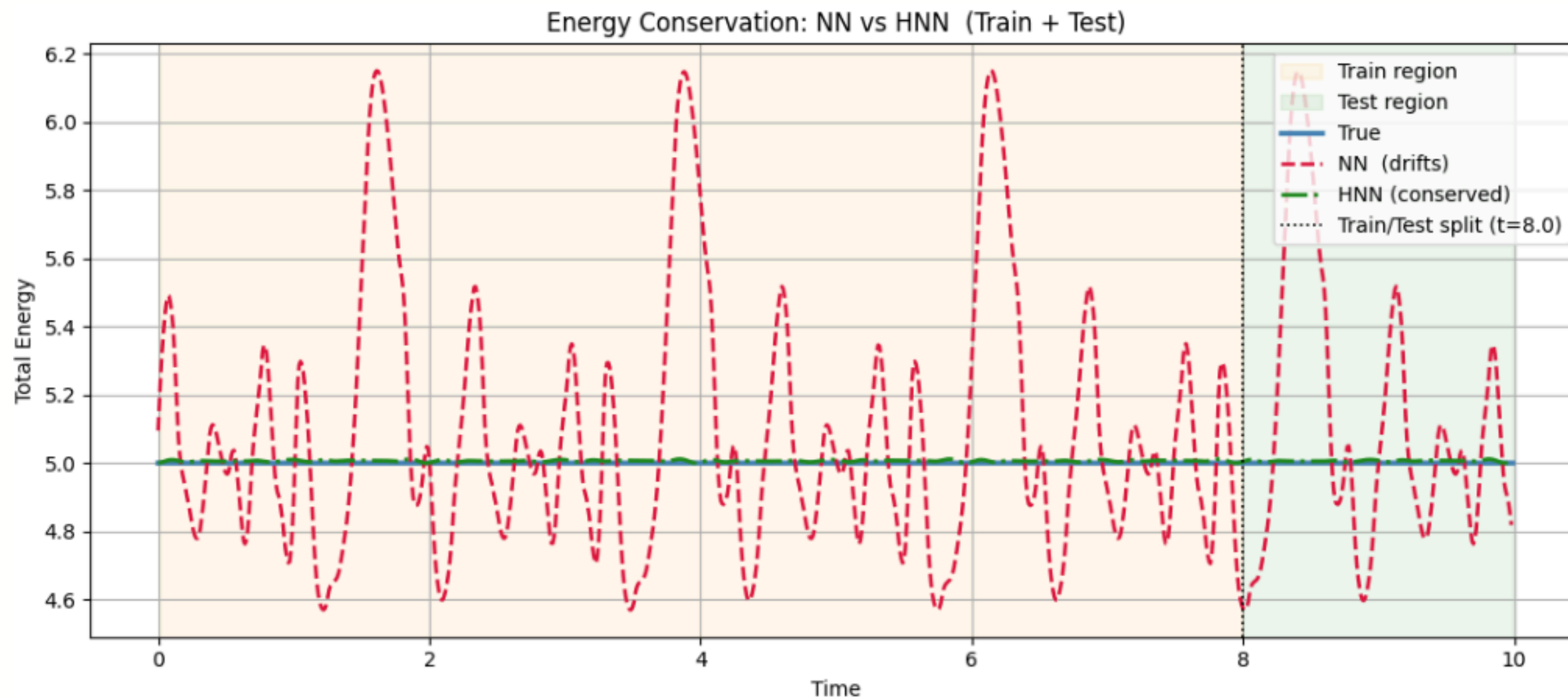
Position-velocity-Energy
Comparison

Energy Conservation Error: NN vs HNN



Energy- Error Comparison

Direct Comparison (NN vs HNN)



Energy Comparison

Key Insights & Challenges

Key Insights:

- Learning physics > fitting data
- Structural constraints improve stability
- HNN captures true system dynamics

Challenges:

- Requires understanding of physics (Hamiltonian)
- More complex training process
- Sensitive to data quality and initialization

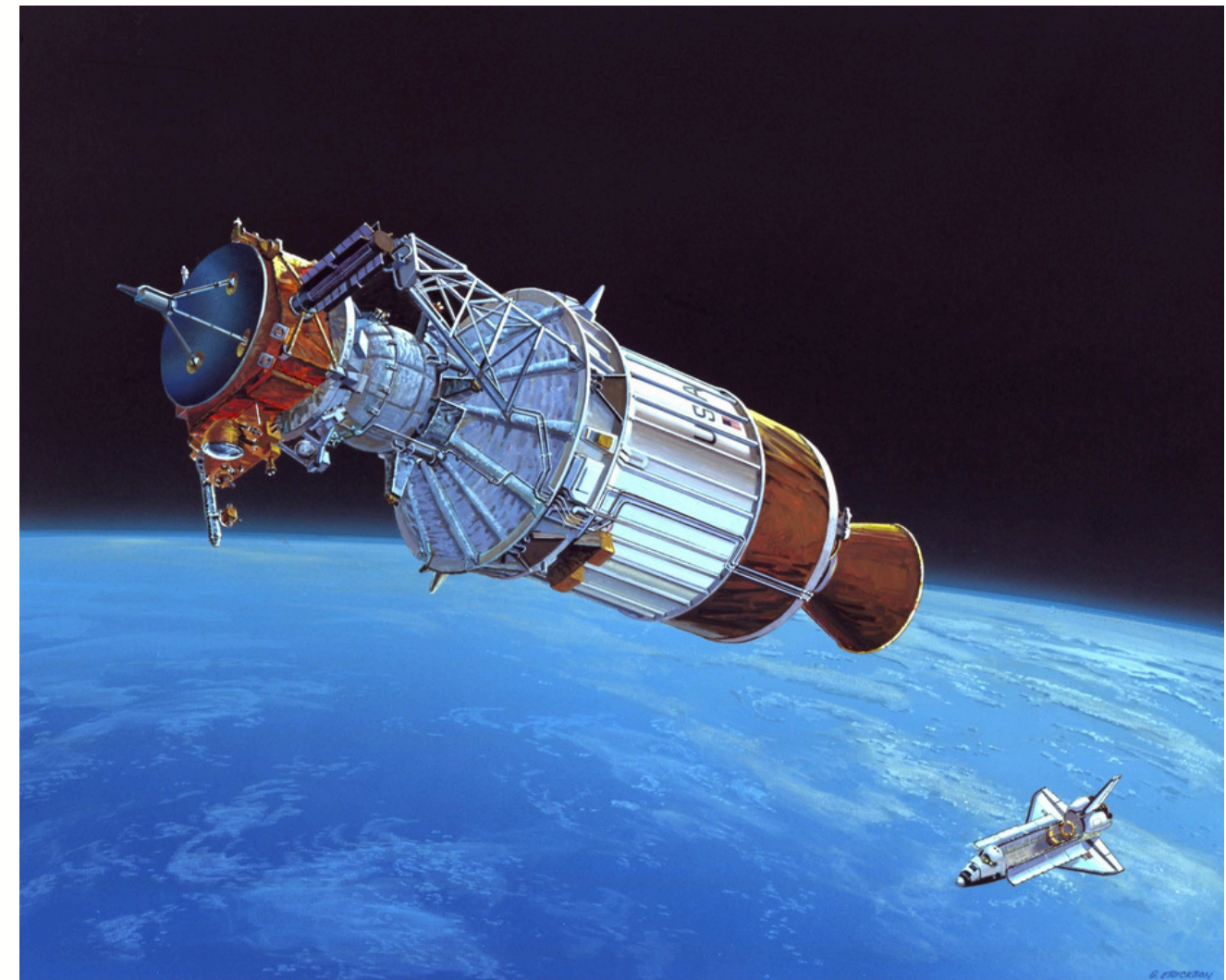
Applications & Future Work

Applications:

- Robotics and control systems
- Physics-based simulations
- Aerospace and orbital dynamics
- Mechanical system modeling

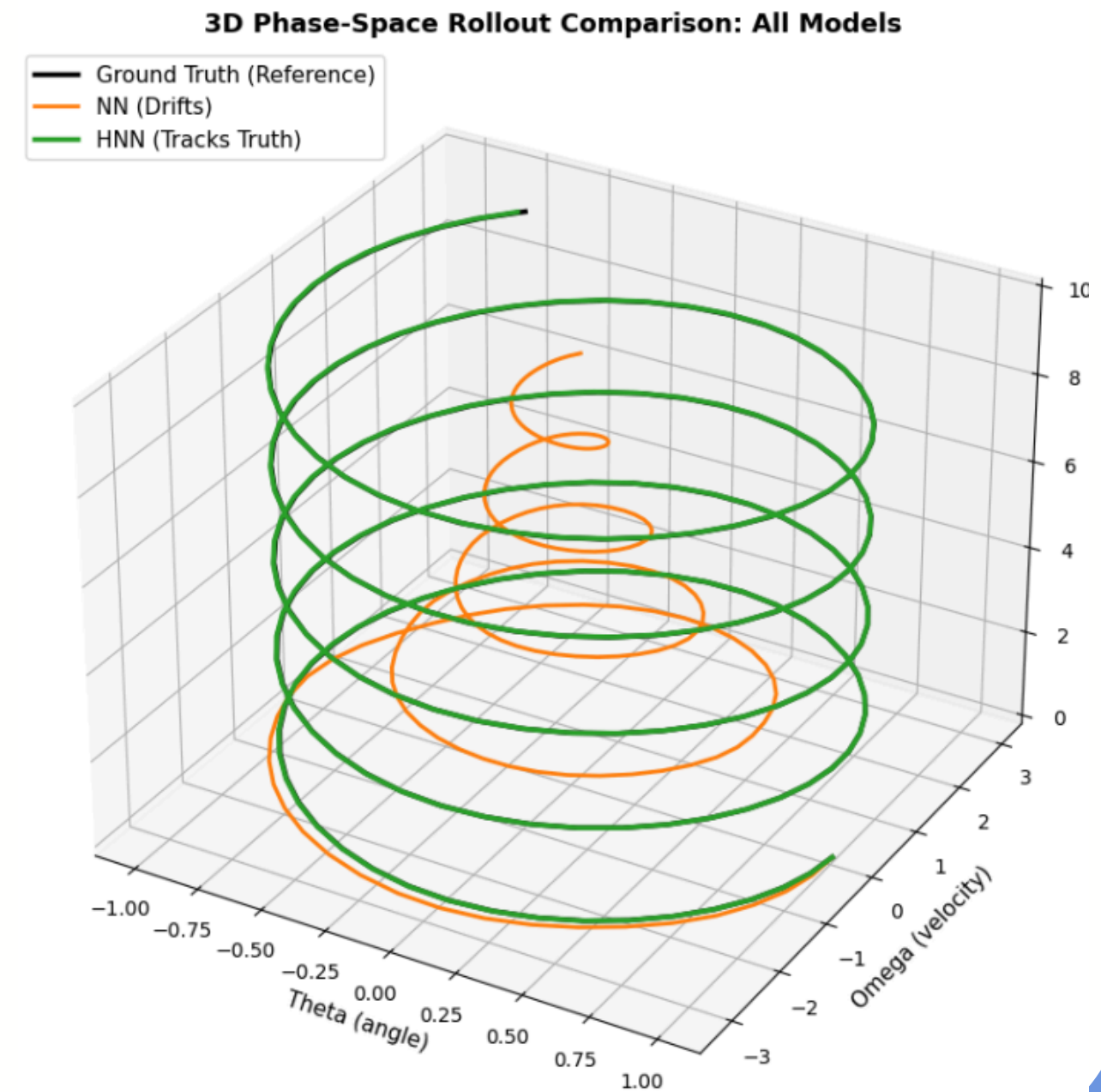
Future Work:

- Extend to multi-body systems
- Handle chaotic dynamics
- Combine with GNN and RL



Conclusion

- Standard NN fails to preserve physical laws
- HNN models system using energy (Hamiltonian)
- Ensures stable and physically consistent predictions
- Better suited for scientific machine learning



References

- Greydanus et al., Hamiltonian Neural Networks, NeurIPS 2019
- PyTorch Documentation
- GitHub repositories for HNN implementations
- John R. Taylor, Classical Mechanics



Thank You

Presented by Amit and Prashant